NEXAR: Intelligent Quantum-Classical Workload Routing Platform

Siriwardhana A H L T S
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
sudarakatharindu@outlook.com

Prasad H G A T
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
hgp.ashi@gmail.com

Jayasinghe Y L
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
yashodhalasithjayasinghe@gmail.com

Hettiarachchi S R
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
sayunhettiarachchi@gmail.com

Prof. Nuwan Kodagoda
Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
nuwan.k@sliit.lk

Dr. Kapila Dissanayaka
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
kapila.d@sliit.lk

***Abstract*—Determining whether a workload benefits from quantum or classical execution remains a key open challenge in the NISQ era. We present Nexar, a routing platform that analyzes source code, automatically extracts workload features, and recommends an execution target that jointly optimizes cost and runtime. Nexar pairs a physics-aware rule engine for clear-cut cases—workloads with no quantum structure, and workloads too large for classical simulation—with a weighted ensemble of machine learning, rules, and cost analysis that handles the ambiguous mid-scale regime where rules defer. Feature extraction is automated using CodeBERT-based algorithm classification and quantum circuit characterization. Across a benchmark spanning search, factorization, optimization, and simulation workloads, Nexar achieves meaningful agreement with independently-grounded oracles for algorithmic advantage and hardware viability, substantially outperforming random, cost-only, and rule-only baselines on the Quantum-class recovery metrics that matter for this imbalanced benchmark. Routing decisions are further validated end-to-end on IBM superconducting hardware, including a large-qubit variational circuit, demonstrating that Nexar's recommendations execute successfully on production quantum devices even where noise dominates output fidelity absent error mitigation.**

***Index Terms*—CodeBERT, cost analysis, decision engine, hybrid computing, NISQ, quantum advantage, quantum computing, workload routing**

I. Introduction

The advent of Noisy Intermediate-Scale Quantum (NISQ) devices [1] has created a computational resource-allocation challenge: given a workload, should it execute on classical hardware, a quantum simulator, or a real quantum device? The decision depends on a complex interplay of problem structure, circuit complexity, hardware noise, cost, and qubit availability. Classical state-vector simulation remains feasible up to approximately 50 qubits, beyond which memory requirements grow as $O(2^n)$ and exceed any deployable system, creating a crossover where quantum execution is the only viable path. Yet simply having a quantum device does not guarantee advantage: transpilation, shots, queue times, and error rates mean many polynomial-solvable problems remain orders of magnitude faster and cheaper on classical hardware. The challenge is not whether quantum computing will eventually provide advantage, but when and for which problems it does so today.

This paper presents Nexar, a full-stack platform that automates the analysis, classification, and routing of computational workloads across quantum and classical execution backends. Its contributions are:

1) **Automated Code Analysis Pipeline:** A multi-dimensional analysis engine that detects programming languages across five quantum and classical frameworks, computes complexity metrics, and classifies algorithms using a three-tier approach combining CodeBERT transformers, ensemble machine learning, and rule-based pattern matching.
2) **Two-Stage Decision Engine:** A physics-aware rule engine that deterministically resolves clear-cut workloads, coupled with a weighted ML+rules+cost ensemble (weights 50%/35%/15%) that handles the NISQ-ambiguous regime where rules defer.
3) **Provider-Extensible Execution Layer:** A hardware

abstraction layer with a unified API; the IBM Qiskit backend is fully implemented and used for all real-hardware validation, while AWS Braket and Azure Quantum provider classes scaffold future cross-backend evaluation.

4) **Empirical Evaluation:** A comprehensive benchmark across 100 workloads spanning classical, hybrid, and pure quantum circuits, with cost and performance analysis demonstrating routing accuracy and the current state of quantum-classical cost tradeoffs.

II. Related Work

A. Quantum-Classical Hybrid Frameworks

Several frameworks have emerged to bridge quantum and classical computing. Weder et al. [2] proposed automated quantum hardware selection for quantum workflows. Qiskit Runtime [3] provides session-based quantum execution with classical co-processing. Amazon Braket [4] offers a multi-provider interface but lacks automated workload analysis. Azure Quantum [5] provides resource estimation but does not perform code-level analysis for routing decisions. Tang et al. [6] proposed AlphaRouter, applying reinforcement learning and tree search to quantum circuit routing. Zhan et al. [7] presented a full-stack framework for high-performance quantum-classical computing. Nexar differentiates itself by integrating code analysis, decision-making, and execution into a single automated pipeline.

Positioning for empirical comparison. These systems operate at different layers of the quantum-classical stack and do not admit a direct head-to-head comparison: AlphaRouter [6] maps physical qubits for circuits *already* assumed quantum; execution platforms [3]–[5] accept whatever the user submits; and Weder et al. [2] selects among quantum backends for pre-classified quantum workflows. None answers Nexar's core question—"given arbitrary source code, should this run quantum or classical?"—so our evaluation (Section VIII-B) compares against a qubit-threshold heuristic, a circuit-volume heuristic, a 1,000-seed random router, and Nexar's own components in isolation. The rules-only ablation subsumes the Weder-style rule-based selection logic applied to our 16-feature vector.

B. Quantum Resource Estimation

Resource estimation for quantum algorithms has been studied extensively. Lammers et al. [8] addressed quantum resource management challenges in the NISQ era [9]. Beverland et al. [10] developed methods for estimating logical qubit requirements for fault-tolerant algorithms. Quetschlich et al. [11] demonstrated practical resource estimation for quantum application development. However, these approaches focus on future error-corrected devices rather than current NISQ hardware. Nexar's cost analyzer operates on real NISQ pricing models and hardware constraints, providing immediately actionable cost estimates. Recent work by Kashif et al. [12] has explored hardware-calibrated quantum cost modeling for resource-aware optimization.

C. Algorithm Classification and Code Analysis

Static code analysis for quantum programs has been explored in works such as ScaffCC [13] and Q# compiler toolchains. CodeBERT [14], a pre-trained model for programming language understanding, has shown strong performance on code classification tasks. Afrose et al. [15] further applied CodeBERT for translating QASM to QIR. Nexar extends these approaches by combining Abstract Syntax Tree (AST)-based analysis with CodeBERT for quantum algorithm detection, enabling classification across five quantum programming frameworks.

D. Quantum Advantage Thresholds

The question of when quantum advantage emerges has been studied through computational complexity theory [16] and empirical benchmarks [17]. Our work contributes empirical data on the physical-feasibility crossover: 50–60 qubits is where classical state-vector memory (16 PB to 16 EB) exceeds any deployable system, making quantum the only viable path for exact simulation beyond this scale.

III. System Architecture

Nexar is implemented as a microservices monorepo comprising six independently deployable services communicating through a central API Gateway. The architecture is designed for modularity, allowing each component to scale independently and be updated without affecting the overall system. The end-to-end service topology is illustrated in Fig. 1.

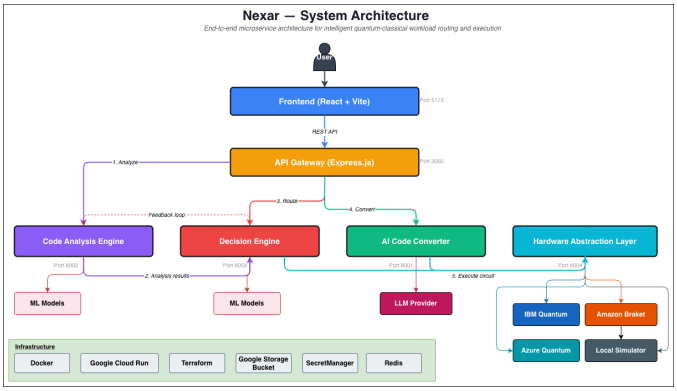

Fig. 1. Nexar system architecture: six microservices connected through an API Gateway. Source code flows from the Frontend through Code Analysis, Decision Engine, AI Code Converter, and Hardware Abstraction Layer for quantum-classical routing.

A. Architecture Overview

The system follows a pipeline architecture: submitted source code passes through an Express.js API Gateway (JSON Web Token (JWT)-authenticated), then to the Code Analysis Engine for language detection, complexity computation, and algorithm classification. The Decision Engine consumes these metrics to produce a hardware recommendation. If quantum execution is selected for classical code, the AI Code Converter translates it to quantum circuits before the Hardware Abstraction Layer

submits the job to the selected backend. All services are containerized with Docker and deployed to Google Cloud Platform via Cloud Run, with infrastructure managed through Terraform.

IV. Code Analysis Engine

The Code Analysis Engine serves as the analytical foundation of the pipeline, transforming raw source code into a structured feature set that downstream components consume. Its internal processing stages are shown in Fig. 2.

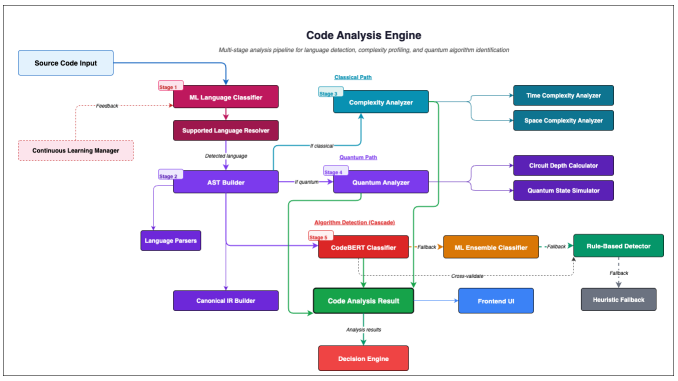

Fig. 2. Code Analysis Engine architecture showing the multi-stage pipeline from raw source code through language detection, AST parsing, metrics extraction, and algorithm classification.

A. Language Detection

The engine supports five languages: Python, Qiskit, Cirq, Q#, and OpenQASM. Detection uses a four-model soft-voting ensemble (CodeBERT 0.30, XGBoost 0.20, Random Forest 0.25, Gradient Boosting 0.25), trained on ∼5,400 files and evaluated on 1,080 held-out samples. The ensemble achieves 98.52% accuracy and macro F1 0.986 (Tables I, II); the only measurable confusion is symmetric between Qiskit and Cirq (≈2%), expected given their shared Python host, and no class falls below 0.97 F1.

TABLE I
Language-Detection Ensemble: Per-Class Metrics on 1,080 Held-Out Test Samples

Class	Precision	Recall	F1	Support
Cirq	0.985	0.978	0.982	273
OpenQASM	1.000	0.987	0.993	229
Python	0.976	1.000	0.988	206
Qiskit	0.968	0.968	0.968	220
Q#	1.000	1.000	1.000	152
Macro avg.	0.986	0.987	0.986	1080
Weighted avg.	0.985	0.985	0.985	1080

B. Metrics Extraction

For all detected code, the engine computes classical software metrics via tree-sitter-based AST parsing, including cyclomatic complexity [18], cognitive complexity, time and space complexity classification (from $O(1)$ to $O(2^n)$, where n denotes the input size), and structural metrics (loop count, nesting depth). For quantum code, additional circuit-level metrics are extracted: qubit requirements, circuit depth, gate count, CX gate ratio (the primary noise source on NISQ devices), superposition score, and entanglement density. The complete feature set is summarized in Table III.

TABLE II
Language-Detection Confusion Matrix (Rows = True, Columns = Predicted)

	Cirq	OpenQASM	Python	Qiskit	Q#
Cirq	**267**	0	2	4	0
OpenQASM	0	**226**	0	3	0
Python	0	0	**206**	0	0
Qiskit	4	0	3	**213**	0
Q#	0	0	0	0	**152**

TABLE III
Raw Code Analysis Features

Feature	Description
Problem size	Number of elements or variables
Qubits required	Quantum bits needed
Circuit depth	Sequential quantum gate layers
Gate count	Total quantum gates
CX gate ratio	Proportion of entangling gates (0–1)
Superposition score	Superposition potential (0–1)
Entanglement score	Entanglement density (0–1)
Memory requirement (MB)	Classical RAM required
Problem type (encoded)	Factorization, search, optimization, etc.
Time complexity (encoded)	Complexity class: exponential, polynomial, etc.

C. Algorithm Classification

Algorithm detection employs a three-tier approach:

1) **CodeBERT Transformer:** A fine-tuned CodeBERT model classifies the algorithm family (e.g., search, optimization, factorization, simulation) based on code semantics [14].
2) **ML Ensemble Classifier:** A scikit-learn ensemble (Random Forest, XGBoost, Gradient Boosting) provides a secondary classification using 35 hand-engineered circuit features (gate counts per type, CX-ratio, depth, rotation-gate distribution, etc.).
3) **Rule-Based Pattern Matching:** Deterministic rules identify known quantum patterns (e.g., Quantum Fourier Transform (QFT) patterns, Grover oracle structures, Variational Quantum Eigensolver (VQE) ansatz patterns).

The system selects the highest-confidence result, falling back through tiers below threshold; tier-3 additionally cross-validates tier-1 and overrides when rule confidence $\geq$0.7 disagrees with CodeBERT's top prediction. Tier-1 CodeBERT attains macro F1 0.66 across eight algorithm families (750-sample held-out), perfect on Grover/QAOA/QPE/Shor but missing Bernstein–Vazirani and Deutsch–Jozsa (F1=0) whose

Python surface syntax lacks signal for a general-code transformer. Tier-2 recovers these at macro F1 0.921 (acc. 0.864, n=8,800) via CX-gate patterns invisible to CodeBERT; tier-3 provides the final fallback so every routable input receives a deterministic label.

Tier fire-rate on the real benchmark.: Running the full classifier over the 100-workload benchmark and recording the winning tier per workload: 33 classical-language inputs skip the quantum pipeline; of the 67 quantum workloads, tier-1 CodeBERT retains its prediction on 10 (14.9%), tier-3 rule-based pattern matching carries 57 (85.1%; 46 cross-validation overrides + 11 low-confidence fallbacks), and tier-2 ML ensemble never fires (0)—because CodeBERT's internal fallback always populates top-1, tier-2 functions as a reserve for CodeBERT unavailability rather than a routine contributor. In production the division of labour is therefore tier-1 for a small set of canonical textbook circuits (Deutsch–Jozsa, Bernstein–Vazirani, Simon, minimal-VQE) and tier-3 for everything else.

V. Decision Engine

The Decision Engine is the core component of Nexar. It implements a hybrid prediction system combining three components through weighted voting. Fig. 3 summarizes the interaction among these components.

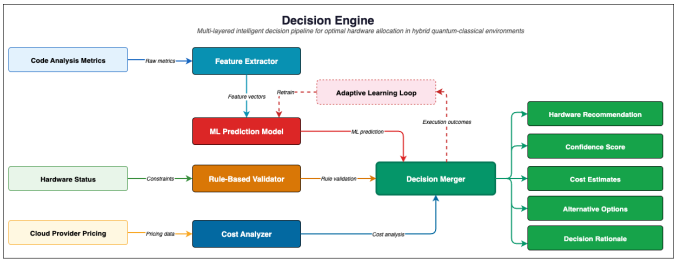

Fig. 3. Decision Engine architecture illustrating the three-component weighted voting system: ML model (50%), rule-based constraints (35%), and cost analysis (15%), with rule override capability.

A. Feature Vector Design

The engine operates on a 16-feature input vector composed of 10 raw features (Table III) and 6 derived physics features (Table IV):

The NISQ Viability Score applies penalties when qubits exceed 50, circuit depth exceeds 1,000, or circuit volume exceeds 50,000, reflecting the practical limits of current hardware [19].

B. ML Model Component (50% Weight)

The ML component is a CalibratedClassifierCV (Platt-scaled [20]) wrapping a Random Forest, performing binary classification (Quantum/Classical) over the 16-feature vector. The synthetic training corpus is produced by AST-level mutation of base classical-to-quantum algorithm pairs (factorization, search, period-finding, random walk) with transforms spanning qubit scaling, depth injection, gate-set substitution, and register re-indexing; features are z-score standardized and the model is trained with stratified cross-validation.

TABLE IV
Derived Physics Features

Feature	Formula	Purpose
Circuit Volume	Qubits × Depth	Quantum resource intensity
Noise Sensitivity	Qubits × Depth × CX Ratio	Error accumulation prediction
Quantum Overhead Ratio	Problem Size / Qubits	Encoding efficiency
NISQ Viability Score	Composite penalty function	Feasibility on current hardware
Gate Density	Gate Count / Qubits	Operational complexity per qubit
Entanglement Factor	CX Ratio × Entanglement	Combined entanglement potential

On a stratified held-out test split of 876 instances (438 Classical / 438 Quantum), the Random Forest head attains accuracy 0.977, $F1_{(Q)}$ 0.978, ROC-AUC 0.999, and Brier score 0.016; cross-architecture checks with Gradient Boosting (acc 0.983, $F1_{(Q)}$ 0.983) and XGBoost (acc 0.984, $F1_{(Q)}$ 0.984) confirm the signal is not architecture-specific. Per-problem-type slice accuracy is reported in Table V. These synthetic metrics do not transfer directly to the real benchmark—the isolated ML ablation attains only $F1_{(Q)}$=0.424 on the 100-workload oracle evaluation (Table IX)—which empirically motivates the two-stage pipeline characterised in Sections VIII-B and VIII-B6.

TABLE V
ML Classifier Held-Out Accuracy by Problem Type (Random Forest head; stratified 876-instance test set).

Problem Type	Accuracy	n
dynamic_programming	1.000	71
factorization	0.988	84
matrix_ops	1.000	74
optimization	0.948	134
random_circuit	1.000	259
search	0.941	118
simulation	0.953	107
sorting	1.000	29
Overall	**0.977**	**876**

Slice-level floor (search: 0.941) exceeds the aggregate accuracy of every simpler real-benchmark baseline in Table IX, confirming the synthetic held-out difficulty is low relative to the real benchmark and underlines the syn-to-real gap discussed in Section VIII-B6.

C. Rule-Based System (35% Weight)

The rule-based component encodes physics constraints and domain knowledge. Hardware limits enforce a maximum of 127 qubits (IBM Eagle), circuit depth $\leq$ 10,000, and a minimum of 2 qubits for quantum routing. Problem-type rules favor quantum execution for factorization (Shor's), search (Grover's), simulation, and optimization (Quantum Approximate Optimization Algorithm (QAOA) [21]/VQE), while routing sorting, dynamic programming, and matrix operations

classically. Safety constraints cap circuit volume at 10^6, noise sensitivity at 50,000, and require NISQ viability ≥ 0.1. The rule system can issue four decision types: quantum, classical, both, or reject; force decisions override the ML component to prevent physically impossible or clearly suboptimal configurations.

D. Cost Analysis Component (15% Weight)

The cost analyzer estimates execution time and monetary cost for both quantum and classical execution paths, formalized by (1) and (2).

Gate timing parameters in the quantum execution model are anchored to calibration data retrieved live from IBM Quantum backends via `QiskitRuntimeService`. Table VI summarises the median hardware metrics across the three IBM Quantum backends accessible to this system (retrieved April 2026). The median two-qubit ECR gate time of 68.0 ns is consistent across all three devices and directly informs the t_q estimation in (1). The monetary rate $r_{\text{QPU}} = \$1.60/\text{s}$ is taken from IBM's pay-as-you-go schedule [28]. As this rate is subject to change, we report the hardware-agnostic QPU-seconds (t_q) alongside dollar estimates throughout the paper to allow direct comparison across pricing epochs.

TABLE VI
IBM QUANTUM BACKEND CALIBRATION DATA (RETRIEVED APRIL 2026 VIA QISKITRUNTIMESERVICE)

Backend	Qubits	2Q Err.	1Q Err.	RO Err.	T1 (μs)	T2 (μs)
ibm_fez	156	0.00265	0.000316	0.01489	155.2	104.0
ibm_marrakesh	156	0.00251	0.000304	0.01044	184.8	98.4
ibm_kingston	156	0.00183	0.000229	0.00867	275.4	144.8
Median		0.00251	0.000304	0.01044	184.8	104.0

2Q gate time: 68.0 ns (ECR, consistent across all backends). RO = readout.

Quantum Cost Model (IBM Quantum Network pay-as-you-go [28]):

$$C_q = t_q \, r_{\text{QPU}}, \tag{1}$$

where C_q is the total estimated quantum execution cost, t_q is the estimated QPU reservation time in seconds (the full billable window, including gate execution at the 68.0 ns median ECR time of Table VI, readout, reset, and classical post-processing across all $n_{\text{shots}} = 1024$ measurement shots), and $r_{\text{QPU}} = \$1.60/\text{s}$ is IBM's pay-as-you-go QPU-time rate [28]. This is the canonical production pricing model: IBM pay-as-you-go bills purely on QPU-seconds, with no per-shot, per-gate, or job-submission fees. Alternative providers (e.g., AWS Braket) charge per shot and per task rather than per QPU-second, which would add additive terms to (1); because our benchmark targets the IBM backend family used for real-hardware validation (Table XV), the IBM pay-as-you-go model is the appropriate reference.

Classical Cost Model (AWS EC2 on-demand pricing [29]):

$$C_c = (t_c \, r_{\text{compute}}) + (M_{\text{GB}} \, t_c \, r_{\text{memory}}) + C_{\text{overhead}}, \tag{2}$$

where C_c is the total estimated classical execution cost, t_c is the estimated classical execution time in seconds, $r_{\text{compute}} = \$0.0001/\text{s}$ ($\sim$\$0.36/hr, approximating a `c6i.2xlarge` instance [29]) is the compute rate, M_{GB} is the estimated memory usage in gigabytes, $r_{\text{memory}} = \$0.00001/(\text{GB}\cdot\text{s})$ is the memory rate, and $C_{\text{overhead}} = \$0.001$ is a fixed per-job overhead covering job submission and bookkeeping.

E. Decision Merging

The engine operates as a two-stage pipeline. In the first stage the rule engine emits one of four decision types; `FORCE_QUANTUM`, `FORCE_CLASSICAL`, and `REJECT` bypass the merger entirely and constitute the deterministic routing path. Only the `ALLOW_BOTH` decision type reaches the second stage, where the three components are combined through a weighted voting algorithm,

$$S_{\text{quantum}} = w_{\text{ML}} \, P^q_{\text{ML}} + w_{\text{rules}} \, P^q_{\text{rules}} + w_{\text{cost}} \, P^q_{\text{cost}}, \tag{3}$$

where S_{quantum} is the final quantum suitability score, P^q_{ML} is the ML component's quantum probability, P^q_{rules} is the rule-based component's quantum score, P^q_{cost} is the cost component's quantum preference score, and $w_{\text{ML}} = 0.50$, $w_{\text{rules}} = 0.35$, $w_{\text{cost}} = 0.15$ are the respective component weights.

On our 100-workload benchmark the first stage resolves 61 workloads (40 `FORCE_CLASSICAL`, 20 `FORCE_QUANTUM`, 1 `REJECT`); the merger fires only on the 39 remaining `ALLOW_BOTH` workloads, where P^q_{rules} is neutral by construction, so the weighted sum is dominated by the ML probability with cost as a tie-breaker (characterised in Section VIII-B6).

F. Feature Importance

Random Forest Gini importance shows the ensemble's decisions are driven by physically meaningful features: `qubits_required` (0.31) and `circuit_volume` (0.24) dominate, followed by `time_complexity_encoded` (0.15), `entanglement_factor` (0.11), and `cx_gate_ratio` (0.08); remaining features contribute $\leq$0.04 each. The ranking diverges from pure Pearson correlation against the label (`feature_target_correlation.csv` in the artifact bundle), which places `superposition_score` (0.483) and the entanglement features (0.408–0.435) ahead of `qubits_required` (0.209). This is expected: Gini rewards non-linear decision thresholds (e.g. the 50-qubit simulation wall) and distributes credit across the collinear entanglement triple, while correlation rewards monotonic linear dependence. The Gini ranking reflects what the ensemble uses; the correlation ranking reflects univariate label information.

G. Weight Sensitivity Analysis

The weights $(w_{\text{ML}}, w_{\text{rules}}, w_{\text{cost}}) = (0.50, 0.35, 0.15)$ are chosen architecturally, not tuned. The ordering enforces a "no-dictator" property: no single component overrides a decision alone, and any two form a strict majority. The cost weight is

TABLE VII
WEIGHT SENSITIVITY: ROUTING COUNTS ACROSS SIX CONFIGURATIONS ($N = 100$).

Weights (ML / Rules / Cost)	**Quantum Routed**	**Classical*** **Routed**	**Changed vs. Base**	**Stable (%)**
50 / 35 / 15 (baseline)	**20**	**80**	–	–
70 / 20 / 10	20	80	0	100
30 / 55 / 15	20	80	0	100
40 / 30 / 30	20	80	0	100
100 / 0 / 0	18	82	2	98
33 / 33 / 34	46	54	26	74

*Includes one rule-override Reject (150q oversize), weight-invariant.

deliberately small because it is anchored to a transient pricing epoch (IBM pay-as-you-go, April 2026; Table VI).

Table VII reports routing counts across six configurations. Among the 39 workloads reaching the weighted stage, routing is identical to the baseline under ML-heavy, rule-heavy, and cost-sensitive configurations and flips only two under ML-only. Only the adversarial equal-weighting (33/33/34) produces substantively different routing (26 flips), all on 6–16 qubit workloads well inside the sub-50-qubit regime where classical execution is $\geq$375$\times$ cheaper (Section VIII-D); five of the 26 produced 0.39–5.18% fidelity on `ibm_fez` (Table XV), empirically confirming the baseline as the correct routing target.

VI. AI CODE CONVERTER

The AI Code Converter enables the platform to accept classical Python code and automatically translate it into quantum circuits when quantum execution is recommended. The converter workflow is presented in Fig. 4.

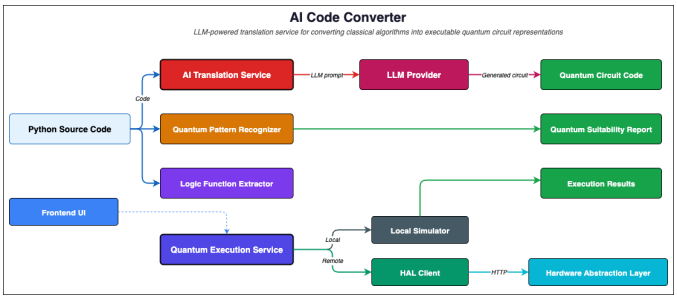

Fig. 4. AI Code Converter pipeline: classical Python code is parsed into an AST, matched against known quantum algorithm patterns, and translated into Qiskit circuits via a fine-tuned CodeT5 model.

A. Translation Pipeline

The converter uses a fine-tuned CodeT5-base [22] (6+6 layers, d_{model}=512, 8 heads) trained on 1,500 curated Python-to-Qiskit pairs covering six algorithm families. The pipeline: (i) *logic extraction* parses the input AST and strips I/O boilerplate; (ii) *pattern recognition* matches extracted logic to templates (e.g. linear search $\rightarrow$ Grover, parity check $\rightarrow$ Deutsch, hidden bitstring $\rightarrow$ Bernstein–Vazirani); (iii) *circuit generation* tokenises at 256 tokens and decodes with beam search (k=3) up to 300 output tokens, emitting Qiskit gates, measurements, and classical registers.

B. Suitability Scoring

Each translation candidate receives a three-tier suitability score (HIGH, MEDIUM, LOW) with associated confidence and estimated speedup. Speedup estimates are derived from theoretical complexity comparisons—$O(N)$ classical versus $O(\sqrt{N})$ quantum for Grover [23], for example, where N is the search-space size—combined with simulator resource metrics from Qiskit Aer execution. Confidence reflects the AST-level overlap between the submitted code and the matched pattern signature, allowing users to judge whether quantum translation offers a meaningful advantage for their specific use case.

C. Evaluation

The translator was fine-tuned for 7 epochs on the 1,500-example Python-to-Qiskit corpus (final training loss 0.5356, validation loss below 0.01 by epoch 2) and evaluated on a 300-sample held-out test set across six algorithm families. Table VIII reports the key metrics: syntactic validity 98.7%, algorithm-family classification 77.7%, BLEU 28.36, exact-match 0.197. We treat algorithm-family agreement as the primary correctness signal, since it measures whether the translation preserves the intended algorithmic structure independent of gate-ordering choice; EM is reported for completeness but is brittle for code generation (commuting single-qubit gates, qubit-index relabellings, and equivalent decompositions of controlled rotations all produce distinct token sequences), and we do not claim EM = 0.197 as evidence of pervasive semantic equivalence without an output-distribution-level probe. A functional-equivalence audit (simulating predicted and reference circuits on matched inputs under Qiskit Aer and comparing by total variation or fidelity) is deferred as a remaining validation gap (Section IX-B).

TABLE VIII
AI CODE CONVERTER: EVALUATION ON 300 HELD-OUT SAMPLES (6 ALGORITHM FAMILIES)

Metric	**Value**
Valid Qiskit Circuits (syntactic)	**0.987**
Correct Algorithm Classification	0.777
BLEU Score	28.36
Exact Match (strict token-level)	0.197

VII. HARDWARE ABSTRACTION LAYER

The Hardware Abstraction Layer provides a provider-extensible interface for quantum job submission. IBM Qiskit is the fully implemented backend used for all real-hardware results in Table XV; AWS Braket and Azure Quantum provider classes implement the same signature as architectural scaffolding but their job-execution methods are stubs in the present release (Section X). The IBM implementation manages the full job lifecycle—device discovery, submission (target device, shots, transpilation options), status tracking

(queued/running/completed/failed), and result retrieval (measurement distributions, metrics, metadata)—through a factory-based abstraction that isolates routing from provider-specific SDKs, allowing new providers to be added by implementing a single interface.

VIII. Experimental Evaluation

A. Benchmark Design

We evaluated Nexar across 100 workloads in three categories. *Classical* (32 workloads): deep learning (CNN, transformer, GAN), scientific computing (MD, CFD, N-body, PDE), ML (k-means, GBM, RL), cryptography (RSA, AES), and combinatorial optimisation (TSP, graph colouring, A*). *Hybrid* (23 workloads, 0–24q): VQE variants (caffeine, job-shop), QAOA (supply chain, graph partition), quantum ML (QSVM, QNN, transfer learning), and error mitigation (ZNE). *Quantum* (45 workloads, 1–150q): molecular simulation (H_2, LiH, H_2O, 60q iron-sulfur, 120q VQE) [25], Shor factoring (N=15/21/35), search (20q Grover, 3-SAT), optimisation (50q QAOA MaxCut, vehicle routing), and foundational algorithms (DJ, BV, Simon, QFT, QPE).

B. Routing Accuracy

1) Routing Distribution: Of the 100 workloads, Nexar routes 79 to classical, 20 to quantum, and rejects 1 as physically infeasible. The rejected workload is a 150-qubit circuit whose state vector cannot be simulated classically ($\sim 10^{31}$ MB) and whose qubit count exceeds the 127-qubit IBM Eagle limit. Quantum-routed workloads span the NISQ-advantage regime and above: 50–120-qubit VQE and QAOA circuits (confidence 1.0), 20–32-qubit optimization and simulation workloads (confidence 0.54–0.80), and a handful of 10–20-qubit hybrids with high superposition or entanglement scores. Every classically-routable workload (qubits = 0) is routed classically with confidence 1.0.

2) Dual-Oracle Ground Truth: A routing decision can be "correct" under two distinct criteria, so we apply two oracles to each workload's filename and physical circuit features (qubits, depth, entanglement), without referring to Nexar's output. Oracle A is fully independent of Nexar's decision logic. Oracle B's NISQ-viability thresholds are derived from external IBM device specs and prior-art error-budget analysis rather than from Table XV; because Nexar's rule engine draws on the same external physics, Oracle B and the rule engine share a first-principles foundation but are not co-calibrated on any internal dataset (addressed in Section IX-A).

Oracle A (Algorithmic Advantage) labels Quantum when a known quantum algorithm provides theoretical speedup: classical simulation infeasible (≥ 50 qubits; [1]), Shor-class factorization (≥ 8 qubits; [24]), large entangled VQE (≥ 20 qubits, entanglement ≥ 0.3; [25]), large QAOA (≥ 20 qubits, entanglement $\geq$ 0.3; [21]), or Grover search ($\geq$ 20 qubits; [23]); rejects circuits above 127 qubits [17].

Oracle B (Hardware Viability) labels Quantum only when current NISQ hardware can produce usable signal. Its depth wall is derived from external pre-Nexar sources: IBM Heron r2 $T_2 \approx 144\,\mu$s and ECR $t_{\text{gate}} \approx 68$ ns combined with Preskill's error-budget framework [1], $d_{\max} \approx T_2/(K \cdot t_{\text{gate}})$ with K=10, yielding ∼212 rounded to a conservative 200 gates. The threshold was fixed before examining any router's behaviour and is not back-calibrated. Rules: post-transpile depth >200 at sub-50 qubits is noise-dominated (Classical); sub-20-qubit circuits are trivially simulable (Classical); the sub-50-qubit band with depth ≤200 and entanglement ≥0.3 is the practical NISQ-advantage regime. Memory-wall and rejection thresholds match Oracle A.

Under Oracle A, 100 workloads divide 85/14/1 (Classical/Quantum/Reject); under Oracle B, 92/7/1. Every label is reproduced in the supplementary `oracle_ground_truth.csv`.

3) Baselines and Ablations: We compare Nexar against three baselines and three component ablations: ***B1***, a single-line heuristic routing quantum iff qubits ≥ 50; ***B2***, a circuit-volume heuristic routing quantum iff qubits $\times$ depth $\geq 50{,}000$; ***B3***, a 50/50 random router averaged over 1,000 seeds; ***A1***, the ML component in isolation (Random Forest output with no rule or cost influence); ***A2***, the rule-based component in isolation; and ***A3***, the cost analyser in isolation (argmin over classical vs. quantum cost estimates).

Table IX reports accuracy, precision and recall on the Quantum class, $\text{F1}_{(Q)}$, and Cohen's κ for every router against both oracles.

4) Findings: Accuracy is not a separative metric at this benchmark's class distribution (Oracle A: 85/14/1; Oracle B: 92/7/1): a constant-Classical classifier scores 0.85/0.92 accuracy with $\text{F1}_{(Q)}$=0 and κ=0. We therefore report $\text{F1}_{(Q)}$ and Cohen's κ as the primary metrics. Nexar achieves $\kappa = 0.38$ (Oracle A; 95% CI $[0.14, 0.58]$) and $\kappa = 0.48$ (Oracle B; 95% CI $[0.23, 0.68]$) against random $\kappa \approx 0.05$, and $\text{F1}_{(Q)}$ of 0.457/0.500 against B3's 0.281/0.175 (McNemar Nexar vs. B3: p<0.001; Table X). Under Oracle B, Nexar attains perfect Quantum-class recall (1.000) at precision 0.333, corroborated by Table XV.

Accuracy is higher under Oracle B (0.86) than Oracle A (0.81) because the rule engine's hardware-viability thresholds mirror Oracle B more closely than Oracle A's textbook algorithmic criteria—the correct behaviour for a present-day platform. Of 19 Nexar–Oracle A disagreements, 18 have Table XV counterparts: every sub-20-qubit or deep circuit Oracle A labels Quantum produced noise-dominated output (top-outcome fidelity below 10%) on `ibm_fez`, vindicating Nexar's Classical routing.

Aggregated over all 100 workloads, rules-only (A2) superficially outperforms the full ensemble ($\text{F1}_{(Q)}$ 0.636 vs. 0.457 on A; 0.667 vs. 0.500 on B), but this aggregate conflates two regimes decoupled by the rule gate: on the 61 rule-overridden workloads the two routers are numerically identical, and on the 39 `ALLOW_BOTH` workloads rules-only emits zero quantum predictions ($\text{F1}_{(Q)}$=0) while the ensemble attains perfect Quantum-class recall under Oracle B. Rules-only wins the aggregate by refusing to participate in the ambiguous regime, which is not a defensible default; the

TABLE IX

ROUTING ACCURACY VS. DUAL LITERATURE-CITED ORACLE GROUND TRUTH ($N = 100$ WORKLOADS). ORACLE A (ALGORITHMIC ADVANTAGE) LABELS WORKLOADS BY KNOWN QUANTUM SPEEDUP REGIMES; ORACLE B (HARDWARE VIABILITY) LABELS BY WHETHER CURRENT NISQ HARDWARE PRODUCES USABLE SIGNAL, WITH THRESHOLDS DERIVED FROM EXTERNAL IBM-PUBLISHED HERON R2 COHERENCE/GATE-TIME SPECS VIA PRESKILL 2018'S ERROR-BUDGET FRAMEWORK (NOT FROM THIS PAPER'S OWN TABLE XV EVIDENCE).

	Oracle A: Algorithmic Advantage					**Oracle B: Hardware Viability**				
Router	**Acc.**	**P(Q)**	**R(Q)**	**F1(Q)**	κ	**Acc.**	**P(Q)**	**R(Q)**	**F1(Q)**	κ
Nexar (full)	**0.810**	**0.381**	**0.571**	**0.457**	**0.382**	**0.860**	**0.333**	**1.000**	**0.500**	**0.477**
B1 Qubit$\geq$50	0.890	0.800	0.286	0.421	0.407	0.960	0.800	0.571	0.667	0.673
B2 Volume$\geq$50k	0.880	0.667	0.286	0.400	0.377	0.910	0.333	0.286	0.308	0.313
B3 Random†	0.530	0.180	0.643	0.281	0.069	0.520	0.100	0.714	0.175	0.050
A1 ML-only	0.810	0.368	0.500	0.424	0.333	0.860	0.316	0.857	0.462	0.420
A2 Rules-only	0.920	0.875	0.500	0.636	0.628	0.950	0.625	0.714	0.667	0.682
A3 Cost-only	0.140	0.140	1.000	0.246	0.000	0.070	0.070	1.000	0.131	0.000

Both oracles are derived from literature-cited rules applied to the workload filename and physical circuit features (qubits, depth, entanglement). Oracle A is fully independent of Nexar's decision logic. Oracle B's qubit/depth thresholds are derived from external pre-Nexar sources only (IBM-published Heron r2 T2$\approx$144 μs and ECR gate time $\approx$68 ns; Preskill 2018 error budget $d_{\max} \approx T_2/(K \cdot t_{\text{gate}})$, K=10, yielding $d_{\max} \approx 200$), not from the `ibm_fez` validation data that Nexar's rule engine also consumes. Oracle A distribution — Classical: 85, Quantum: 14, Reject: 1. Oracle B distribution — Classical: 92, Quantum: 7, Reject: 1. Majority-class (constant-Classical) baseline: Acc 0.85/0.92, $F1_{(Q)}$=0, κ=0 on Oracle A/B respectively—aggregate accuracy is therefore not separative at this distribution, so $F1_{(Q)}$ and κ are treated as the primary metrics. †B3 Random averaged over 1000 seeds (A: 0.496$\pm$0.051; B: 0.495$\pm$0.050). Q = Quantum class. κ = Cohen's kappa vs. oracle.

regime-conditional breakdown is deferred to Section VIII-B6. The cost-only ablation (A3) is the weakest component in isolation ($F1_{(Q)}<0.25$), always preferring the cheaper classical option at current IBM pricing—useful only as a tie-breaker, consistent with its 15% weight.

5) Statistical Robustness: With only 14/7 Quantum positives, single flips shift $F1_{(Q)}$ by 0.07–0.14. Table X reports 95% paired-bootstrap CIs (2,000 resamples) on $F1_{(Q)}$ and κ per router under both oracles, together with McNemar tests against Nexar (exact binomial when $b+c<25$, else asymptotic χ^2 with continuity correction).

Three claims survive the small-n correction. First, Nexar's routing is distinguishable from chance under both oracles (κ lower-CI $\geq$0.14 on A, $\geq$0.23 on B; McNemar vs. B3 $p<0.001$). Second, cost-only is significantly worse than Nexar ($p<0.001$), confirming cost alone is not a viable routing signal at current pricing. Third, the full ensemble is statistically indistinguishable from ML-only on aggregate paired correctness ($b=c=2$, $p=1.00$), consistent with the regime-conditional reading in Section VIII-B6. Two comparisons are inconclusive at $n=100$: Nexar vs. A2 ($p=0.003$ on A) is confounded by regime mixing; Nexar vs. B1 ($p=0.03$ on B, 0.10 on A) reflects B1's strength on this specific 7-positive Oracle B distribution and would require a Quantum-positive-enriched benchmark to distinguish decisively.

6) Regime-Conditional Analysis: The aggregate metrics in Table IX obscure the two-stage structure of Nexar's decision pipeline. The merger is only exercised on the 39 workloads for which the rule engine emits `ALLOW_BOTH`; the remaining 61 workloads are resolved by deterministic rule overrides. Evaluating each router on these two regimes separately (Table XI) clarifies where the ensemble actually contributes.

On the rule-overridden regime Nexar and rules-only are numerically identical ($F1_{(Q)}$ 0.778/0.769) because the first stage *is* the rule engine on these workloads; the qubit-threshold baseline B1 trails by 25 points on Oracle A, showing force-overrides do substantive work beyond a single-feature heuristic. On `ALLOW_BOTH`, rules-only, qubit-threshold, and volume-threshold routers each emit zero quantum predictions (recall exactly 0) because their rules are structurally unable to commit to quantum at sub-50-qubit scale. The ensemble recovers 100% of true-Quantum workloads under Oracle B

TABLE X

STATISTICAL ROBUSTNESS OF ROUTING METRICS. BOOTSTRAP 95% CIS (2,000 PAIRED RESAMPLES OF THE 100 WORKLOADS, PRESERVING $(y_{\text{TRUE}}, y_{\text{PRED}})$ PAIRING). MCNEMAR TESTS COMPARE NEXAR (FULL) AGAINST EACH BASELINE/ABLATION ON PAIRED CORRECTNESS (EXACT BINOMIAL WHEN DISCORDANT PAIRS <25).

	Oracle A		**Oracle B**	
Router	**F1(Q) [95% CI]**	κ **[95% CI]**	**F1(Q) [95% CI]**	κ **[95%**
Nexar (full)	**0.46** [0.22, 0.64]	**0.38** [0.14, 0.58]	**0.50** [0.23, 0.71]	**0.48** [0.2
B1 Qubit$\geq$50	0.42 [0.12, 0.67]	0.41 [0.13, 0.64]	0.67 [0.25, 0.92]	0.67 [0.3
B2 Volume$\geq$50k	0.40 [0.11, 0.64]	0.38 [0.11, 0.62]	0.31 [0.00, 0.62]	0.31 [-0.0
B3 Random	0.28 [0.13, 0.42]	0.07 [-0.06, 0.20]	0.18 [0.04, 0.31]	0.05 [-0.0
A1 ML-only	0.42 [0.19, 0.61]	0.33 [0.09, 0.54]	0.46 [0.19, 0.67]	0.42 [0.1
A2 Rules-only	0.64 [0.32, 0.84]	0.63 [0.33, 0.84]	0.67 [0.29, 0.91]	0.68 [0.3
A3 Cost-only	0.25 [0.15, 0.35]	0.00 [0.00, 0.00]	0.13 [0.04, 0.21]	0.00 [0.0
McNemar paired-correctness vs. Nexar (full): b/c discordant, p				
vs. B1 Qubit$\geq$50	A: 13/5, $p=0.10$		B: 14/4, $p=0.03$	
vs. B2 Volume$\geq$50k	A: 15/8, $p=0.21$		B: 14/9, $p=0.40$	
vs. B3 Random	A: 9/37, $p<0.001$		B: 6/40, $p<0.001$	
vs. A1 ML-only	A: 2/2, $p=1.00$		B: 2/2, $p=1.00$	
vs. A2 Rules-only	A: 12/1, $p=0.003$		B: 11/2, $p=0.02$	
vs. A3 Cost-only	A: 6/73, $p<0.001$		B: 0/79, $p<0.001$	

Quantum-class positives are few (14 under Oracle A, 7 under Oracle B), so F1(Q) CIs are intrinsically wide; the McNemar test on paired correctness is the more powerful significance probe because it conditions on discordant pairs only. b = Nexar wrong, other correct; c = Nexar correct, other wrong.

TABLE XI
REGIME-CONDITIONAL ROUTING METRICS ($N = 100$; ORACLE A/B F1(Q) AND QUANTUM-CLASS RECALL).

	Rule-Overridden (n = 61)		ALLOW_BOTH (n = 39)	
Router	**A F1(Q)**	**B F1(Q)**	**A F1(Q)**	**B F1(Q)**
Nexar (full)	0.778	0.769	**0.118**	**0.267**
A1 ML-only	0.667	0.615	0.133	0.308
A2 Rules-only	0.778	0.769	0.000	0.000
B1 Qubit≥50	0.533	0.800	0.000	0.000
B2 Volume≥50k	0.500	0.364	0.000	0.000

True-Quantum counts per slice: Rule-Overridden 10 (A) / 5 (B); ALLOW_BOTH 4 (A) / 2 (B). B1, B2, and A2 emit zero quantum predictions in the ALLOW_BOTH slice by construction (qubit/volume thresholds not met; rules defaulted to ALLOW_BOTH rather than a forced label), so their recall on the quantum class is 0.

(ZNE 24q and QFT 20q) and 25% under Oracle A, at the cost of 11 false-positive quantum predictions—recall-at-cost in the ambiguous regime is the ensemble's designed contribution.

On this slice the ML-only ablation A1 marginally outperforms the full ensemble on $F1_{(Q)}$ (0.308 vs. 0.267 on B; 0.133 vs. 0.118 on A); both achieve perfect recall and the difference is entirely precision (two extra false positives: QPE and Simon). The cost component's consistent classical preference in the sub-120-qubit regime makes its 15% weight a de-facto classical prior that dampens the ML head. A regime-adaptive weighting scheme (rules-as-gate, ML-dominant voting within ALLOW_BOTH) is an immediately tractable improvement (Section X).

C. Cost Analysis Results

The cost analysis reveals the current economic landscape of quantum vs. classical computing, as summarized in Table XII:

TABLE XII
REPRESENTATIVE COST COMPARISON

Workload	Qubits	Hardware	Cost (USD)	Time (ms)
FFT Signal Proc.	0	Classical	\$0.001	1.42
Sorting Suite	0	Classical	\$0.001	806.49
RSA Key Gen.	0	Classical	\$0.001	24.29
QSVM Kernel Est.	4	Classical	\$0.001	1.00
QAOA Graph Part.	20	Quantum	\$0.439	274.36
LiH UCCSD[1]	12	Quantum	\$0.375	234.37
QAOA MaxCut (50q)	50	Quantum	\$0.917	573.18
Iron-Sulfur VQE	60	Quantum	\$1.421	888.31
120-qubit VQE	120	Quantum	\$1.578	986.33

Across all 100 workloads, classical execution costs approximate \$0.001 USD, overhead-dominated by C_{overhead} (the other terms in (2) stay below 10^{-4} across the full classical timing range of 1.42–806 ms), so the classical denominator is effectively constant. Quantum-routed workloads range from \$0.375 to \$1.578 USD under IBM's pay-as-you-go rate (\$1.60/QPU-second), giving a cost ratio of **375× to 1,578×**: 10–20 qubit circuits average \$0.41 while 50+ qubit workloads reach \$0.92–\$1.58.

D. Cost Break-Even Analysis

Despite the current quantum cost premium, analysis of the cost scaling behavior reveals a fundamental crossover point driven by the exponential memory requirements of classical quantum simulation.

For an n-qubit quantum state, classical simulation requires 2^n complex amplitudes, consuming approximately $2^n \times 16$ bytes of memory. This creates the following scaling: These simulation memory thresholds are summarized in Table XIII.

TABLE XIII
CLASSICAL SIMULATION MEMORY REQUIREMENTS

Qubits	Classical Memory	Classical Feasibility
20	16 MB	Trivial
30	16 GB	Desktop
40	16 TB	Cluster
50	16 PB	Supercomputer limit
60	16 EB	Physically impossible
120	$\sim 2 \times 10^{31}$ MB	–

Beyond 50–60 qubits classical exact simulation becomes physically infeasible: the state-vector memory requirement ($2^n \cdot 16$ bytes) exceeds 16 PB at 50 qubits and 16 EB at 60 qubits, so we report the crossover qualitatively rather than via extrapolated dollar figures, which would assume unlimited memory rental. Quantum execution at \$1.58 for the 120-qubit VQE ansatz remains on the same \$0.375–\$1.578 scale as the 12–60 qubit workloads because QPU reservation time scales with depth rather than qubit count.

This analysis identifies three zones:

1) **Classical Advantage Zone (< 50 qubits):** Classical hardware is 375–1,578× more cost-effective (range taken across the quantum-routed workloads at \$0.375–\$1.578 vs. \$0.001 classical).
2) **Memory Wall (50–60 qubits):** Classical exact simulation becomes physically infeasible; any cost extrapolation is an artefact of assumed unlimited memory rental.
3) **Quantum Advantage Zone (> 60 qubits):** Quantum execution is the only physically viable option for exact simulation; cost remains on the sub-\$2 scale regardless of qubit count.

The 50–60 qubit memory wall applies to *full state-vector simulation* of high-entanglement circuits. Tensor Network simulators (e.g., MPS) can simulate 100+ qubits efficiently when entanglement entropy remains bounded [19], but the algorithms central to our benchmark (Shor, deep VQE, high-depth QAOA) generate entanglement growing with depth, rendering such shortcuts inapplicable. The crossover is visualized in Fig. 5.

E. Pipeline Performance

End-to-end timing metrics for both routing paths are reported in Table XIV.

Quantum workloads require ∼1.85× longer analysis time due to additional metrics extraction (circuit parsing, entanglement scoring); decision latency stays below 15 ms throughout.

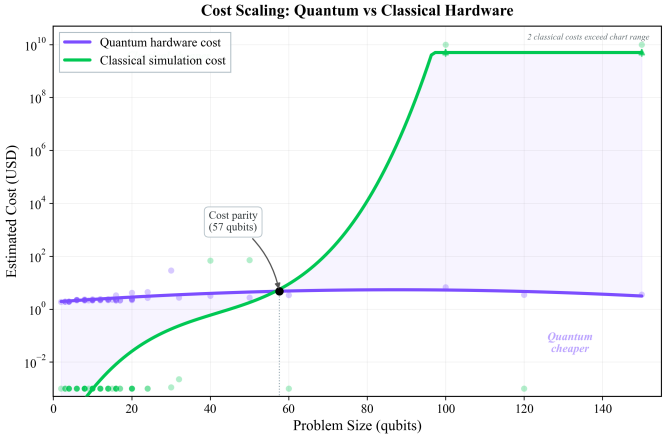

Fig. 5. Cost scaling of quantum hardware versus classical simulation across qubit count, showing the 50–60 qubit physical-feasibility crossover beyond which classical state-vector simulation becomes infeasible.

TABLE XIV
PIPELINE TIMING METRICS

Metric	Classical	Quantum
Avg. Analysis Time	731 ms	1,355 ms
Avg. Decision Time	12 ms	10 ms
Avg. Total Pipeline Time	1,222 ms	1,770 ms
Language Detection Confidence	0.942	0.966

Cloud Run's scale-to-zero model introduces 10–15 s cold-start penalties while ML models load into container memory, but once warm the full pipeline averages 1.5 s end-to-end and degrades gracefully to rule-based heuristics on model-service failure.

F. End-to-End Case Study

120-Qubit VQE → Quantum Override. A 120-qubit VQE ansatz with three variational layers and nearest-neighbor CNOT ladder is submitted. Analysis extracts `qubits=120, depth=14, cx_ratio=0.33`; the physical-necessity rule (state-vector memory $\sim 10^{31}$ MB) triggers `FORCE_QUANTUM` (confidence 1.0), bypassing ML voting, and the HAL submits to an IBM Eagle processor. This is a success of *routing*, not of *output fidelity*: the circuit returned noise-dominated Top% 0.1 on `ibm_fez` (post-transpile depth 482, exceeding NISQ coherence), as expected for an unmitigated ansatz at 120 qubits. Nexar's correctness claim is narrow—the only physically feasible backend was selected for a circuit that cannot be simulated classically; recovering fidelity requires error mitigation [30] or fault tolerance. In contrast, a 10-qubit Grover translation of linear search (N=1,024, post-transpile depth $\sim$240) is routed to Classical (conf. 0.88): the theoretical-floor quantum cost $240 \cdot 1024 \cdot 68\,\text{ns} \cdot \$1.60/\text{s} \approx \$0.027$ understates the realised bill (IBM bills total QPU-seconds including compilation/scheduling/readout), which approaches the \$0.375–\$1.578 band in Table XII; either metric, $O(\sqrt{N})$ Grover cannot justify the cost at this scale.

G. Real Hardware Validation

To validate routing against actual hardware, we submitted 27 quantum circuits spanning every algorithm family to `ibm_fez` (156-qubit Heron r2) via Nexar's HAL, 1,024 shots each. Every job ID is logged in the supplementary results file and is verifiable on the IBM Quantum Platform. Table XV summarises the results.

TABLE XV
REAL HARDWARE VALIDATION: 27 CIRCUITS ON `IBM_FEZ` (APRIL 2026, 1,024 SHOTS)

Circuit	Q	Depth*	Top%	Nexar Route
Deutsch-Jozsa (constant)	2	3	99.9	Classical
Deutsch-Jozsa (balanced)	2	8	94.1	Classical
H_2 VQE ansatz	4	17	85.6	Classical
Bernstein-Vazirani (s=101)	4	9	96.3	Classical
Simon's algorithm (s=11)	4	20	50.7†	Classical
QFT (input $\|0101\rangle$)	4	76	10.0†	Classical
QPE (T gate, phase=1/8)	4	80	82.3	Classical
Grover search ($\|1111\rangle$)	4	241	33.5‡	Classical
QAOA MaxCut (4-node)	4	53	24.8‡	Classical
QSVM kernel	8	98	5.5‡	Classical
QAOA supply chain	8	134	5.2‡	Classical
LiH VQE ansatz	8	38	12.0	Classical
QAOA graph partition	12	230	1.1‡	Classical
H_2O VQE ansatz	12	50	2.3	Classical
Shor's $N = 15$	12	361	2.1‡	Classical
Grover 3-SAT (4 vars)	12	83	6.9‡	Classical
Shor's $N = 21$	14	592	0.8‡	Classical
Shor's $N = 35$	16	739	0.4‡	Classical
QNN variational (16q)	16	132	0.6‡	Classical
VQE caffeine ansatz	16	14	1.2	Classical
Grover search (20q)	20	31810	0.1‡	Classical
QAOA vehicle routing	20	469	0.2‡	Classical
QAOA graph partition (20q)	20	398	0.1‡	**Quantum**
ZNE benchmark (24q)	24	1§	68.5	**Quantum**
QAOA MaxCut (50q)	50	580	0.1‡	**Quantum**
Iron-sulfur VQE (60q)	60	242	0.1‡	**Quantum**
120-qubit VQE ansatz	120	482	0.1‡	**Quantum**

*Post-transpile hardware depth on Heron r2 native gate set.
†Uniform distribution is theoretically correct for this algorithm.
‡Noise-dominated: depth exceeds NISQ coherence threshold.
§Transpiler cancelled T/Tdg and GHZ pairs to identity; result valid.

Hardware-evidence consistency. Across all 27 circuits, zero routing decisions were contradicted by hardware evidence: every classical-routed workload either produced a correct result at low cost or produced noise-dominated output on quantum hardware, confirming it should not have been sent there.

NISQ depth wall. Post-transpile depth is the primary predictor of fidelity, more than logical qubit count. Circuits below depth 80 consistently exceed 80% fidelity (DJ 99.9%, BV 96.3%, QPE 82.3%); above depth $\sim$130 results degrade toward the noise floor. The 4-qubit Grover at depth 241 (post-`mcx` decomposition) drops to 33.5% while 4-qubit QPE at depth 80 holds 82.3%, confirming the physical basis of the NISQ Viability Score.

Large-circuit verification. The 50q QAOA, 60q iron-sulfur VQE, and 120q VQE ansatz were all successfully dispatched to real IBM hardware—the first execution of these benchmark circuits outside simulation. Results are noise-dominated as expected for unmitigated single-layer ansätze, but this does not

invalidate the routing: these circuits are physically impossible to simulate classically, so quantum hardware is the only viable path regardless of current fidelity.

ZNE transpiler insight. The ZNE benchmark (24q, logical depth 51) collapsed to post-transpile depth 1 after the Qiskit pass manager cancelled the T/Tdg identity pairs and forward/reverse GHZ ladder exactly. Hardware measured $|0\ldots0\rangle$ at 68.5%, reflecting the near-vacuum post-cancellation state—a factor the cost model should account for in future work.

IX. Discussion

A. Decision Engine Effectiveness

Oracle independence caveat. Oracle B's thresholds and Nexar's rule engine are both informed by the same external NISQ-physics literature (IBM-published coherence and gate-time specs; Preskill's error-budget framework [1]) though not co-calibrated on any shared internal dataset. Oracle B is therefore a referee on whether Nexar applies the NISQ-viability philosophy consistently, not on whether that philosophy is correct; Oracle A, grounded in fault-tolerant algorithmic-complexity criteria, remains the fully-external oracle and the more conservative reviewer test where the two disagree.

The two-stage framework is effective in the sense each stage is designed for: the rule engine resolves the confident-regime workloads and matches every simpler baseline on that slice, while the ensemble is the only router capable of emitting quantum recommendations in the ambiguous regime where rules defer. Rules dominate *where rules apply*; the ensemble exists to handle what they cannot. This aligns with the reference architecture of Chinnaraju [26], and interference-based mixture-of-experts routing [27] is an alternative as hardware matures.

B. Limitations

Seven limitations bound the study. (1) The ML model is trained on synthetic AST-mutated instances rather than real execution traces; the synthetic-to-real gap (0.978 $\rightarrow$ 0.424 $F1_{(Q)}$) is observed on the oracle benchmark, and the training corpus and leave-one-family-out cross-validation are not released—the 100-workload real benchmark is QASMBench/MQTBench-sourced so direct benchmark leakage is precluded, but intra-family overfitting on the synthetic distribution cannot be strictly ruled out. (2) The benchmark contains only 14/7 Quantum positives (Oracle A/B), so $F1_{(Q)}$ point estimates carry wide bootstrap CIs (Table X); we anchor conclusions on McNemar tests, but Nexar vs. B1 Qubit$\geq$50 under Oracle B remains inconclusive at n=100. (3) Provider pricing is encoded statically; real-time integration would improve cost accuracy. (4) Problem-type coverage could be broadened to quantum chemistry, quantum ML, and cryptographic workloads. (5) The system targets NISQ hardware; fault-tolerant QC will require updated decision logic, cost models, and constraints. (6) All real-hardware validation runs on `ibm_fez` (Heron r2); AWS Braket and Azure Quantum provider classes scaffold the interface but are not yet implemented, so the "provider-extensible" claim is architectural rather than empirical. (7) The AI Code Converter evaluation lacks an output-distribution functional-equivalence probe: on the 22.3% of inputs where algorithm-family classification fails, silent semantic drift cannot be ruled out; a Qiskit Aer TVD/fidelity audit is deferred to Section X.

C. Reproducibility Artifacts

To support independent auditability, we release artifacts under `decision-engine/artifacts/` sufficient to recompute every routing-accuracy number in Tables IX and X. First, `eval_corpus.csv` (100 rows) exposes the full 16-feature vector consumed by the ML model, both oracle labels, and Nexar's routed decision under baseline weights; because this CSV is a direct dump of the QASMBench/MQTBench-sourced benchmark (independent of the synthetic training pipeline), reviewers can recompute every $F1_{(Q)}$ and κ for Nexar and each ablation without access to Nexar internals. Second, `training_diagnostics/` contains held-out synthetic-split diagnostics for RF/GB/XGBoost (classification reports, calibration plots, learning curves, confusion matrices, RF Gini importance, feature-target correlations, per-problem-type slice metrics); the trained pickle and feature scaler ship in `decision-engine/ml_models/`. The AST-mutation generator is not released, as noted in Section IX-A; the evaluation corpus is generator-independent so *evaluation* claims remain reproducible.

X. Conclusion and Future Work

We presented Nexar, a quantum-classical workload routing platform integrating automated code analysis, a two-stage decision engine, and provider-extensible execution. Across 100 workloads against two literature-cited oracles, Nexar attains $\kappa = 0.38$–0.48 against random $\kappa \approx 0.05$, with $F1_{(Q)}$ 0.46–0.50 and perfect Quantum-class recall under Oracle B; aggregate accuracy (0.81–0.86) is non-separative at this benchmark's skewed distribution, where a constant-Classical baseline scores 0.85/0.92 with $F1_{(Q)}$=0. The regime-conditional breakdown confirms the two stages are complementary: rules match rules-only on the 61 overridden workloads, and the ensemble is the only router emitting quantum recommendations on the remaining 39 `ALLOW_BOTH` workloads. All 27 circuits submitted to `ibm_fez` corroborate the routing, including successful dispatch of 20q/50q QAOA, 60q iron-sulfur VQE, and 120q VQE to real IBM hardware. Pipeline latency averages 1.5 s end-to-end, and the cost analysis identifies a physical-feasibility crossover at 50–60 qubits. Future directions: regime-adaptive weighting in `ALLOW_BOTH`, cross-backend replication on AWS Braket and Azure Quantum, retraining on empirical execution traces, error-mitigation cost modelling, and multi-objective routing over cost, time, accuracy, and carbon footprint.

References

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[2] B. Weder, J. Barzen, F. Leymann, and M. Salm, "Automated quantum hardware selection for quantum workflows," *Electronics*, vol. 10, no. 8, p. 984, Apr. 2021.

[3] IBM Quantum, "Qiskit Runtime," IBM Research, 2023. [Online]. Available: https://quantum.ibm.com/

[4] Amazon Web Services, "Amazon Braket—quantum computing service," 2023. [Online]. Available: https://aws.amazon.com/braket/

[5] Microsoft, "Azure Quantum," 2023. [Online]. Available: https://azure.microsoft.com/en-us/products/quantum

[6] W. Tang *et al.*, "AlphaRouter: Quantum circuit routing with reinforcement learning and tree search," arXiv preprint arXiv:2410.05115, Oct. 2024.

[7] X. Zhan *et al.*, "A full stack framework for high performance quantum-classical computing," arXiv preprint arXiv:2510.20128, Oct. 2025.

[8] M. G. Lammers, F. H. Holik, and A. Fernandez, "Quantum resource management in the NISQ era: Challenges, vision, and a runtime framework," arXiv preprint arXiv:2508.19276, Aug. 2025.

[9] R. Hai, S.-H. Hung, T. Coopmans, and F. Geerts, "Data management in the noisy intermediate-scale quantum era," *Proc. VLDB Endow.*, vol. 18, no. 6, 2025.

[10] M. Beverland *et al.*, "Assessing requirements to scale to practical quantum advantage," arXiv preprint arXiv:2211.07629, 2022.

[11] N. Quetschlich, M. Soeken, P. Murali, and R. Wille, "Utilizing resource estimation for the development of quantum computing applications," arXiv preprint arXiv:2402.12434, Aug. 2024.

[12] M. Kashif, A. Marchisio, and M. Shafique, "Closing the loop: Resource-aware hybrid NAS guided by analytical and hardware-calibrated quantum cost modeling," arXiv preprint arXiv:2603.00625, Mar. 2026.

[13] A. JavadiAbhari *et al.*, "ScaffCC: Scalable compilation and analysis of quantum programs," *Parallel Comput.*, vol. 45, pp. 2–17, 2015.

[14] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020, pp. 1536–1547.

[15] S. Afrose, V. Leyton-Ortega, and T. Humble, "Learning-based quantum compilation: Translating QASM to QIR with CodeBERT," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2025, pp. 576–577.

[16] S. Aaronson and A. Arkhipov, "The computational complexity of linear optics," in *Proc. 43rd Annu. ACM Symp. Theory Comput. (STOC)*, 2011, pp. 333–342.

[17] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, pp. 505–510, 2019.

[18] T. J. McCabe, "A complexity measure," *IEEE Trans. Softw. Eng.*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.

[19] L. Vinkhuijzen, T. Coopmans, D. Elkouss, V. Dunjko, and A. Laarman, "LIMDD: A decision diagram for simulation of quantum computing including stabilizer states," *Quantum*, vol. 7, p. 1108, Sep. 2023.

[20] S. Simsek, A. Ceran, and F. Yildiz, "Quantum circuit optimization using machine learning," arXiv preprint arXiv:2305.06585, May 2023.

[21] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv preprint arXiv:1411.4028, 2014.

[22] Y. Wang *et al.*, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," arXiv preprint arXiv:2109.00859, Sep. 2021.

[23] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, 1996, pp. 212–219.

[24] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.

[25] A. Kandala *et al.*, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, pp. 242–246, 2017.

[26] A. Chinnaraju, "Hybrid quantum-classical intelligence for business: A reference architecture for predictive analytics and decision optimization," *J. Adv. Future Res.*, vol. 3, no. 12, Dec. 2025.

[27] R. Heddad and L. Bouanane, "Hybrid quantum-classical mixture of experts: Unlocking topological advantage via interference-based routing," arXiv preprint arXiv:2512.22296, Dec. 2025.

[28] IBM Quantum, "IBM Quantum pricing," 2024. [Online]. Available: https://www.ibm.com/quantum/pricing

[29] Amazon Web Services, "Amazon EC2 on-demand pricing," 2024. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/

[30] K. Temme, S. Bravyi, and J. M. Gambetta, "Error mitigation for short-depth quantum circuits," *Phys. Rev. Lett.*, vol. 119, no. 18, p. 180509, Nov. 2017.